

Digital Electronics

Part 2 – Boolean Variables and Logic

Gates

Stéphane Le Goff

2026

2. Boolean Variables and Logic Gates

• Boolean Algebra

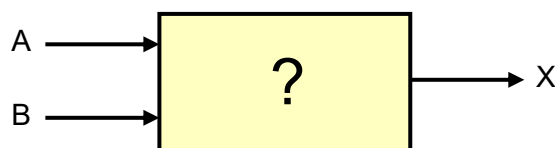
Boolean algebra differs from ordinary algebra in the sense that Boolean constants and variables can only take two values: 0 or 1. Booleans 0 and 1 do not represent actual numbers but instead represent the state of a voltage variable, or what is called its logic level. Some common representations of 0 and 1 are shown in the following table.

Logic 0	Logic 1
False	True
Off	On
Low	High
No	Yes
Low voltage	High voltage

In Boolean algebra, there are three basic logic operations: OR, AND, and NOT. These operations are implemented using logic gates which are in practice simple electronic circuits constructed from transistors.

• Truth Table

A truth table describes how the output of a logic circuit depends on the logic levels present at the input. In the following two-input logic circuit, the table lists all possible combinations of logic levels present at inputs A and B along with the corresponding output level X.



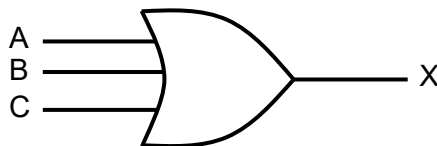
A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

When either input A OR B is 1, the output X is 1. Therefore the "?" in the box is a two-input OR gate. This example provides us with the opportunity to introduce our first logic gate: the OR gate.

• OR Gate

The OR operation produces a result of 1 when any of the input variables is 1, and it produces a result of 0 only when all the input variables are 0. In other words, the gate output is 1 if the first input is 1 OR the second input is 1 OR the third input is 1 and so on, depending on how many inputs this gate has.

As an illustration, the symbol of a three-input OR gate as well as its truth table are shown below.



A	B	C	X
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

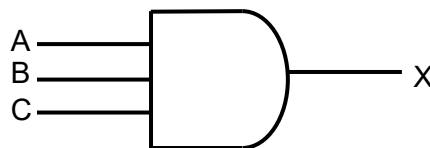
The corresponding Boolean expression of a three-input OR gate is $X = A + B + C$.

Note that the notation adopted for the OR expression is identical to that of the addition in decimal algebra, but this should not confuse us as an OR function does, in fact, have nothing to do with a decimal addition.

• AND Gate

The AND operation produces a result of 1 only when all the input variables are 1. The output is 0 for any case where one or more inputs are 0. In other words, the gate output is 1 if the first input is 1 AND the second input is 1 AND the third input is 1 and so on, depending on how many inputs this AND gate has.

As an illustration, the symbol and truth table of a 3-input AND gate are shown below.



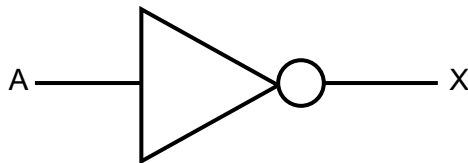
A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

The corresponding Boolean expression is $X = ABC$ or $X = A \cdot B \cdot C$.

Note that the notation adopted for the AND expression is identical to that of the multiplication in decimal algebra.

- **NOT Gate (Inverter)**

The NOT operation is performed on a single input variable. Its symbol and truth table are depicted below.



A	X
0	1
1	0

The corresponding Boolean expression is $X = \bar{A}$.

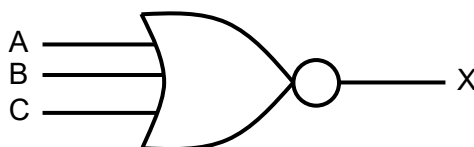
- **Other Logic Gates**

We also have several other logic functions that are of great importance in digital electronics. They are described below.

- **NOR (NOT-OR) Gate**

The NOR gate, which can be seen as an OR gate followed by an inverter, can have as many inputs as desired. For instance, in the case where there are three inputs, its Boolean expression is given by

$$X = \overline{A + B + C}.$$

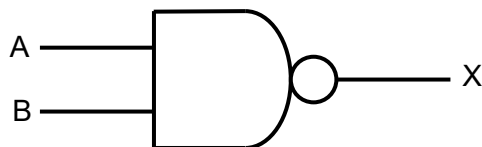


A	B	C	X
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

• NAND (NOT-AND) Gate

The NAND gate, which is equivalent to an AND gate followed by an inverter, can have as many inputs as desired. For instance, in the case where there are two inputs, its Boolean expression is given by

$$X = \overline{AB} \text{ or } X = \overline{A \cdot B}.$$



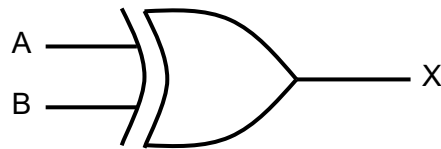
A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

• XOR (Exclusive OR) Gate

The XOR gate only has two inputs. This gate is a bit more complex to implement than those we have studied so far. The Boolean expression of a two-input XOR gate is

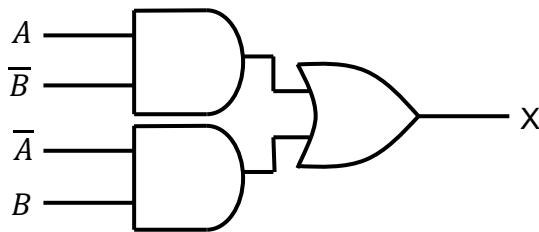
$$X = A \oplus B = A\bar{B} + \bar{A}B.$$

Its symbol and truth table are shown below.



A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

The Boolean expression indicates that an XOR gate can be designed using two AND gates and one OR gate, as shown below. To be completely accurate, we should also have included in this diagram the two inverters needed to generate $\bar{A}$ and $\bar{B}$.

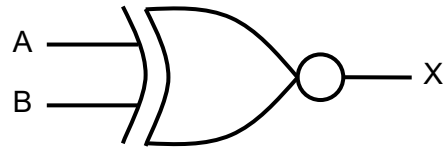


• XNOR (Exclusive NOR) Gate

The XNOR gate has two inputs only and can be viewed as an XOR gate followed by an inverter. Therefore, its Boolean expression is given by

$$X = \overline{A \oplus B} = AB + \bar{A}\bar{B}.$$

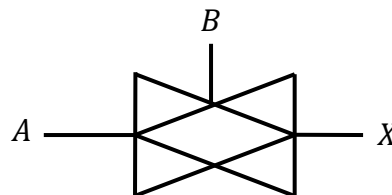
Its symbol and truth table are shown below.



A	B	X
0	0	1
0	1	0
1	0	0
1	1	1

• Transmission Gate

The transmission gate behaves as a switch in digital circuits. Its symbol and truth table are shown below.



B	X
0	Z
1	A

This strange truth table indicates that the output X equals Z if the input B equals 0, whereas it is equal to the input A if B = 1. The value Z refers to the “high impedance” state: it is equal to neither 0 nor 1. Instead, it is left as a floating output, not connected to any node in the circuit.

Transmission gates are very useful for designing a wide range of important logic operators, such as tri-state logic gates (with three possible output values, namely 0, 1, and Z), flip-flops, and multiplexing circuits.

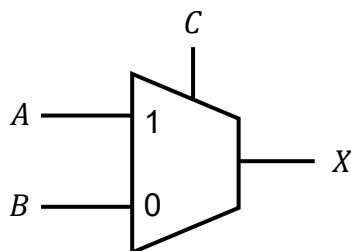
For instance, a two-bit multiplexer (MUX) is a circuit whose output X is given by the Boolean expression $X = AC + B\overline{C}$, where A and B denote the data inputs and C designates the control input. The truth table of such multiplexer is shown below.

C	X
1	A
0	B

This is a very important component in many circuits as it represents the hardware implementation of the conditional instruction often encountered in software.

Two-input multiplexer	Software equivalent
If ($C = 1$)	if (condition = true) then
Output $X = A$	do something
else	else
Output $X = B$	do something else
	endif

The symbol of a two-bit multiplexer is shown below.



A two-bit multiplexer can be designed using an inverter, two two-input AND gates, and a two-input OR gate, or using an inverter and three two-input NAND gates. Another simpler way to design it is through the use of one inverter and two transmission gates, as shown below.

